

Problem A. Iron Warrior

Except at the very beginning, the draw pile always contains at least one card: the card that has just been played. We may therefore assume that the draw pile is already non-empty.

After playing Pommel Strike, if Shrug It Off is not played immediately, then eventually all cards in the hand are exhausted and the play cannot continue forever. Similarly, after playing Rage or Body Slam, the next card must be Pommel Strike. Thus, except for the beginning and the end of the game, the play pattern is forced to repeat

Rage/Body Slam, Pommel Strike, Shrug It Off.

There are only $O(1)$ possibilities for the exceptional prefix and suffix, so they can be enumerated directly. For the main repeated part, the structure implies that there is a prefix where Rage is played and a suffix where Pommel Strike is played; the boundary between them can be found by binary search. The implementation needs big integers.

Problem B. Little, Cyan, Fish!

If every operation affected a whole row, column, or diagonal, the problem would become a 3SUM-like convolution problem and could be solved with one FFT.

For interval operations, use square-root decomposition. Split the $n \times n$ grid into $(n/B) \times (n/B)$ blocks of size $B \times B$. A line intersects $O(n/B)$ whole blocks and $O(B)$ scattered cells. Whole blocks are handled by the same convolution idea, while scattered cells are added by brute force. This gives

$$O\left(n\sqrt{n \log n}\right)$$

time.

The extra logarithm can be removed. While sweeping horizontally, keep the point values of the row polynomials; changing one coefficient updates all needed point values in $O(B)$ time. The same idea applies to columns and diagonals. Choosing $B \approx \sqrt{n}$ gives $O(n\sqrt{n})$ time.

Problem C. Currency

For each i , choose one of the two horizontal edges between $(1, i)$ and $(1, i + 1)$, or between $(2, i)$ and $(2, i + 1)$. Choosing either edge has its own cost. If the choices for positions i and $i + 1$ lie on different rows, then an additional cost for the vertical edge between $(1, i + 1)$ and $(2, i + 1)$ must be paid.

Together with the m constraints, this is a small graph-cut model. Build the corresponding flow network and compute a minimum cut; the side of each vertex in the cut encodes which row is chosen.

Problem D. Widely Known Problem 2

This is the CountDistinct query of the Internal Dictionary Matching problem. Assign a weight v_t to every substring t of the text, initially 0, and add 1 to the weights of all pattern strings. For a query substring $s[l, r]$, the answer is the sum of v over all distinct substrings of $s[l, r]$.

Consider the trie formed by all strings, with characters added in the direction matching the forward suffix automaton. Let f_u be the sum of v over all strings on the path from u to the root. The relevant nodes

form a connected subtree containing the root, so the answer can be written as

$$\sum_{u \text{ is a leaf}} f_u - \sum_u (\# \text{children of } u - 1) f_u.$$

For the second term, scan the right endpoint r . Use a Link-Cut Tree on the suffix automaton to maintain maximal consecutive segments whose last occurrence is the same. During an access operation, when such a segment is met, the endpoint at which its number of trie children increases is determined by the condition

$$l \leq \text{last} - \text{len}.$$

Thus the update over possible l is a range add, and the query is a point query. This part takes $O(n \log^2 n + q \log n)$ time.

For the leaf term, first observe that every leaf has left endpoint l . If $s[l, t]$ is not a leaf, then it has another occurrence $s[l', r'] = s[l, t]$ with $t < r' \leq r$. This gives a threshold x for which the leaf contribution is reduced to sums of the form

$$\sum_{i=x}^r f_{s[l, i]}.$$

After taking differences, the remaining queries are exactly the Count query of the IDM problem, which is solvable with the basic substring structure in $O((n + q) \log n)$ time.

The total complexity is $O(n \log^2 n + q \log n)$. When n and q have the same order, the logarithmic factor can be improved further.

Problem E. Light Drinking and Low Singing

Maintain all maximal runs of 0 and all maximal runs of 1 whose length is at least 2.

For an operation of type 1, every 0-run moves one position to the left and every 1-run moves one position to the right. Operation type 2 is symmetric. The only problematic case is a collision between a 0-run and a 1-run; such a collision can be repaired by direct local modification.

Each collision decreases the total length of long runs by 2. The total length is initially at most n , and operations can only create a linear number of new boundary events. Hence the total amount of brute-force repair over all operations is linear.

Using balanced sets to maintain the runs gives total time complexity $O((n + q) \log n)$.

Problem F. Trash Problem

Enumerate the left boundary of the rectangle and sweep the right boundary. While sweeping, keep for every row whether the current cell is one half of an unfinished 2×2 block or is already completed. If a row is half-filled and the next cell is black, then every interval containing that row is invalid. Otherwise, when a white cell is met, the state of that row is toggled.

At any sweep position, an interval of rows is valid if and only if:

- all row states inside it are completed;
- it does not contain a row that was marked invalid by the previous case;
- for every previously appearing consecutive segment of state 1, its intersection length with the interval is even.

The third condition can be hashed. The parity of intersections of $[l, r]$ with all state-1 segments is even exactly when the set of left endpoints whose intersection parity with $[1, r]$ is odd equals the corresponding set for $[1, l - 1]$. Therefore, while scanning r , hash this set and count equal hashes. With $O(1)$ expected hash-table operations, the total time complexity is $O(n^3)$.

Problem G. Analysis

An operation of cost A can be regarded as adding a heavy edge. We may perform all such operations first, and only then ask how to traverse the resulting graph. After all heavy edges have been chosen, the required number of cost- B operations is

$$\frac{\#\{\text{odd-degree vertices}\}}{2} - 1,$$

unless there are no odd-degree vertices, in which case it is 0.

Therefore the remaining task is a tree DP deciding where to add heavy edges. A state records the parity of the degree of the subtree root and whether the subtree already contains an odd-degree vertex. The transitions are local over children.

The time complexity is $O(n)$.

Problem H. Algebra

Rewrite the k -th power of a subtree size as the number of ways to choose k points in that subtree. Equivalently, introduce k new vertices $n + 1, n + 2, \dots, n + k$, and let each of them choose a parent uniformly among vertices $1, 2, \dots, n$. For each original vertex u , count the number of such parent choices for which all k new vertices are inside the subtree of u .

Process the original vertices from front to back. Let $d[i][j]$ be the number of states after processing the first i vertices, where j not-yet-seen child subtrees still contain at least one of the k new vertices. When a new vertex is processed, there are three transition types:

1. its subtree contains none of the new vertices;
2. it contains at least one new vertex and is attached below a vertex that still has another such unseen child subtree;
3. it contains at least one new vertex and is attached below a vertex for which it is the last such child subtree.

To obtain each vertex answer, run this DP forward once and the transposed process backward once, then combine the two parts at the vertex. The time complexity is $O(nk)$.

Problem I. Twenty-Two

First ignore all useless operations. Global minimum operations can be assumed to be applied in increasing order of their value: if a larger minimum operation appears before a smaller one, the larger operation can be moved next to the smaller one and remains useless.

Thus every interval maximum operation can be placed before any global minimum operation. Its value c may either stay unchanged or become one of the smaller global-minimum values. After this transformation,

we first apply the smallest global minimum, then all interval maximum operations. The problem is to count how many final arrays can be obtained.

Use an interval DP. A typical state is $dp(l, r, x)$, which counts final arrays on $[l, r]$ in which all values are at least x , while the two neighboring positions $l - 1$ and $r + 1$ are less than x . This formulation ensures that only maximum operations fully contained in $[l, r]$ need to be considered. The transition enumerates the minimum value in the interval and splits around it.

When n, m, k are of the same order, the time complexity is $O(n^4)$.

Problem J. Loving You in My Humble Way

Replace each hyperedge (x, y, z) by the three undirected edges (x, y) , (y, z) , and (z, x) . The condition is equivalent to requiring that the resulting graph contain no 4-cycle.

Use the Erdős-Renyi graph ER_q :

- vertices are the distinct lines in the three-dimensional vector space over $\mathbb{F}_q$;
- two vertices are adjacent when the corresponding lines are perpendicular.

This graph has $q^2 + q + 1$ vertices and $\frac{1}{2}q(q + 1)^2$ edges, and it has no 4-cycle.

Construct the hyperedges of the answer as all triangles of ER_q . The absence of 4-cycles gives the required intersection property, and the number of hyperedges satisfies the needed lower bound.

Problem K. Ying's Cup

After fixing one vertex to be a local minimum, all comparisons between it and its neighboring vertices are determined.

We use the classical formula for forests: if every parent must be smaller than every child, then the number of permutations respecting the partial order is

$$n! \prod_u \frac{1}{\text{size}_u}.$$

Run a tree DP. Let $d[u][i][j]$ describe the subtree of u , where i vertices have been fixed as local minima and the current component of the comparison forest has size j . Whenever a new local minimum is selected, it must be smaller than all its children and also smaller than its parent. The latter condition is handled by inclusion-exclusion: count the unrestricted case and subtract the case where the parent is smaller.

The transition is a two-dimensional knapsack. By replacing one dimension with point evaluation and using interpolation, the complexity can be reduced to $O(n^3)$.